

RoadSOS+ Research & Implementation Documentation

1. Problem Statement & Motivation

Road safety management traditionally suffers from severe lag times between hazard formation (e.g., potholes, broken signals) and municipal resolution. This lag is primarily caused by a reactive methodology—authorities wait for fatal accidents to occur before identifying a "danger zone."

The Problems Addressed:

1. **Lack of Real-Time Data:** Citizens lack a unified, immediate platform to report non-injury infrastructural hazards (near-misses).
2. **Resource Misallocation:** Authorities struggle to triage hundreds of incoming reports without manual review, leading to delayed emergency responses.
3. **Reactive vs. Proactive Intervention:** Municipalities lack automated geospatial intelligence to predict where accidents are statistically likely to occur next.

The RoadSOS+ Solution:

RoadSOS+ solves these problems by providing a full-stack, crowdsourced platform. It empowers citizens to report hazards with GPS accuracy and images, utilizes AI to instantly triage and classify reports, and generates dynamic "Risk Zones" that cluster historical data to predict future hotspots—enabling proactive infrastructural maintenance.

2. Technology Stack & Architectural Justification

2.1 The Chosen Stack

- **Frontend:** Next.js 14 (App Router), React, TailwindCSS, Zustand, React-Leaflet
- **Backend:** FastAPI (Python 3.12), Uvicorn, Pydantic V2
- **Database:** PostgreSQL 15, SQLAlchemy 2.0 (ORM), Alembic (Migrations)
- **AI & Storage:** Google Gemini AI (LLM), Cloudinary (CDN)
- **Deployment:** Docker, Railway (Backend), Vercel (Frontend)

2.2 Why This Stack? (Architectural Decisions)

Next.js over standard Create React App (CRA):

- *Reasoning:* Next.js provides robust Edge Middleware for route protection (auth checking before rendering), seamless API proxying, and App Router layouts that drastically reduce re-renders for dashboard interfaces.

FastAPI over Django/Flask/Node.js:

- *Reasoning:* RoadSOS+ relies heavily on I/O bound operations (Database queries, Image Uploads to Cloudinary, and external LLM calls to Google Gemini). FastAPI's native `asyncio` support and `BackgroundTasks` allow the server to accept an image, immediately return a `202 Accepted` to the client, and process the heavy AI inference in the background without blocking the event loop. Django/Flask are synchronous by default, and Node.js lacks the robust scientific/AI ecosystem of Python.

PostgreSQL over MongoDB:

- *Reasoning:* Incident data is highly relational. A user (Citizen) creates an Incident, which requires strict foreign key relationships, ENUMs for statuses (`pending`, `verified`), and cascading deletes for media. NoSQL databases (like MongoDB) would require complex manual joins for the Authority dashboard analytics. Postgres also allows future migration to PostGIS for advanced geospatial bounding-box queries.

Zustand over Redux:

- *Reasoning:* Redux introduces massive boilerplate for simple JWT storage and GPS coordinate persistence. Zustand provides a lean, hook-based state management solution that integrates natively with `localStorage`.
-

3. Implementation Details

3.1 Authentication & Security (JWT + RBAC)

Authentication is stateless to allow for horizontal scaling.

- **Implementation:** Passwords are mathematically hashed using `bcrypt` via `passlib`. Upon login, FastAPI issues a cryptographically signed JSON Web Token (JWT) with a 24-hour expiration.
- **Role-Based Access Control (RBAC):** The token encodes the user's role (`citizen` or `authority`). FastAPI dependency

injection (Depends (get_current_user)) intercepts requests to protected endpoints, decrypts the token, and rejects unauthorized roles with a 401/403 status before business logic executes.

3.2 AI Integration & Triage Automation

Traditional reporting systems require an admin to manually read a description to determine priority.

- **Implementation:** When a citizen uploads an incident, the backend triggers a `BackgroundTask`. The raw text and Cloudinary image URL are streamed to the **Google Gemini AI**.
- **Prompt Engineering:** The LLM is strictly prompted to return a JSON schema classifying the `incident_type` (e.g., `collision`, `road_hazard`) and `severity` (`low`, `medium`, `high`). This structured telemetry is saved back to the database, allowing the Authority Dashboard to automatically sort tickets by AI-determined urgency.

3.3 Dynamic Geospatial Mapping (React-Leaflet)

- **Challenges Solved:** Leaflet manipulates the browser's `window` object. Because Next.js pre-renders code on the server (SSR), importing Leaflet causes fatal crashes (`window is not defined`).
- **Implementation:** We implemented dynamic React wrappers (`next/dynamic` with `ssr: false`), forcing the Leaflet components to hydrate strictly on the client. The map binds to the `locationStore` Zustand state, which utilizes `HTML5 navigator.geolocation` to plot the user and pull nearby incidents via `Axios` interceptors.

3.4 The Risk Engine (Data Clustering)

The core analytical feature is the automated generation of Risk Zones.

- **Implementation:** A scheduled backend service scans the `incidents` and `near_miss_reports` tables. It calculates geometric distances between data points. When a high density of high-severity incidents or non-injury near-misses cluster within a 500m radius, the algorithm calculates a normalized `risk_score`.
- **Impact:** This clustered data is cached in the `risk_zones` table, preventing expensive $O(N^2)$ distance calculations on every map load, and visualizes immediate danger zones for Citizens in real-time.

3.5 Idempotent Cloud Deployment

- **Implementation:** Deployed via Docker on Railway, the backend features a robust `start.sh` entrypoint. It utilizes a Python retry-loop to block execution until the Postgres socket is available. It then executes `alembic upgrade head` to apply schema mutations, queries the `information_schema` to verify table creation, and idempotently seeds the database (only if the user cardinality is 0). This prevents duplicate data errors during frequent cloud scale-ups/restarts.